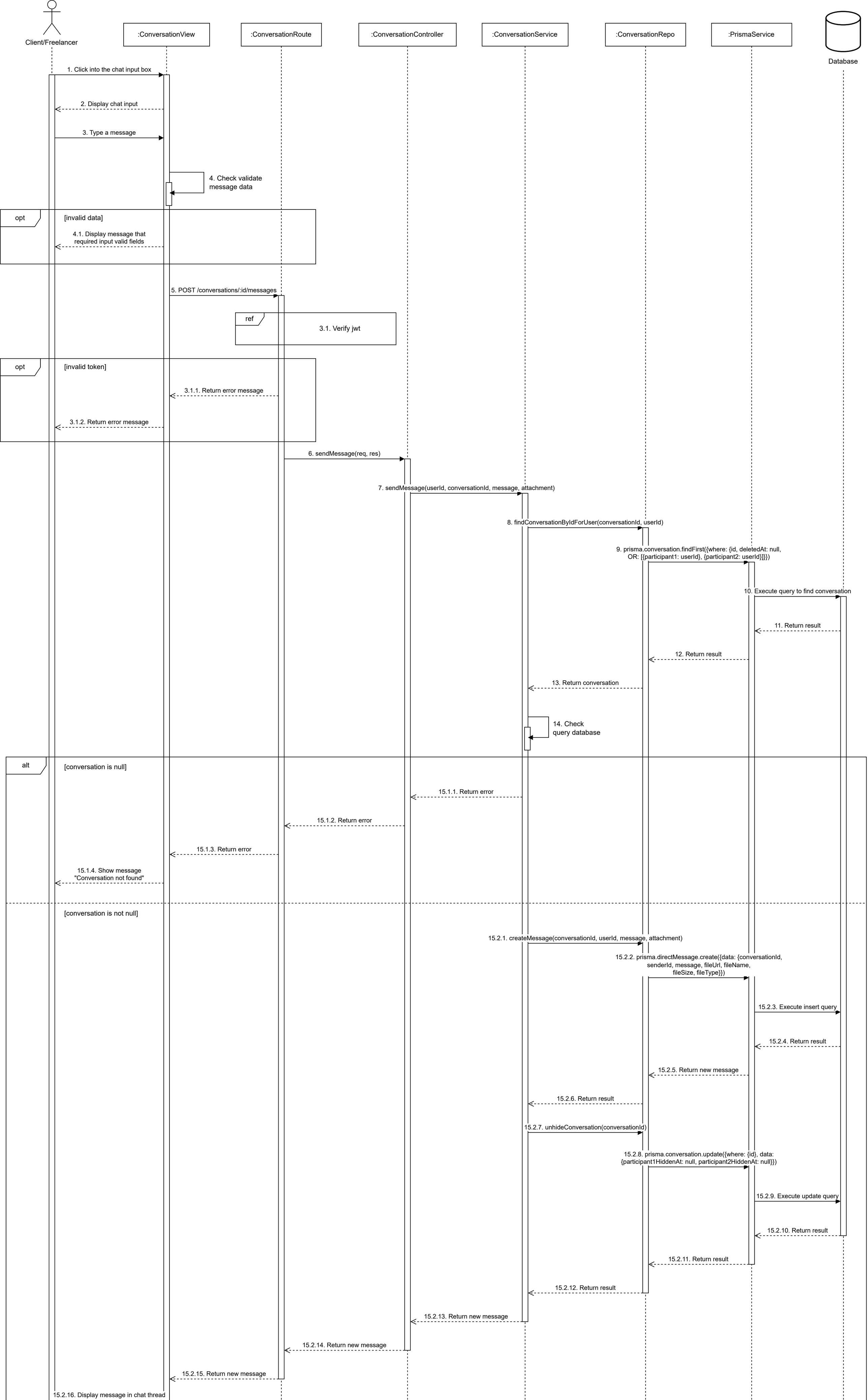

Client/Freelancer
:ConversationView
:ConversationRoute
:ConversationController
:ConversationService
:ConversationRepo
:PrismaService
Database
1. Click into the chat input box
2. Display chat input
3. Type a message
4. Check validate message data
opt
[invalid data]
4.1. Display message that required input valid fields
5. POST /conversations/:id/messages
ref
3.1. Verify jwt
opt
[invalid token]
3.1.1. Return error message
3.1.2. Return error message
6. sendMessage(req, res)
7. sendMessage(userId, conversationId, message, attachment)
8. findConversationByIdForUser(conversationId, userId)
9. prisma.conversation.findFirst({where: {id, deletedAt: null, OR: [{participant1: userId}, {participant2: userId}]}})
10. Execute query to find conversation
11. Return result
12. Return result
13. Return conversation
14. Check query database
alt
[conversation is null]
15.1.1. Return error
15.1.2. Return error
15.1.3. Return error
15.1.4. Show message "Conversation not found"
[conversation is not null]
15.2.1. createMessage(conversationId, userId, message, attachment)
15.2.2. prisma.directMessage.create({data: {conversationId, senderId, message, fileUrl, fileName, fileSize, fileType}})
15.2.3. Execute insert query
15.2.4. Return result
15.2.5. Return new message
15.2.6. Return result
15.2.7. unhideConversation(conversationId)
15.2.8. prisma.conversation.update({where: {id}, data: {participant1HiddenAt: null, participant2HiddenAt: null}})
15.2.9. Execute update query
15.2.10. Return result
15.2.11. Return result
15.2.12. Return result
15.2.13. Return new message
15.2.14. Return new message
15.2.15. Return new message
15.2.16. Display message in chat thread (and deliver realtime to the other user)